Neural Networks Essentials

Autoencoders — From Foundations to Frontiers

Denis C. Ilie-Ablachim

April 22, 2026

Outline

Motivation and Intuition

Mathematical Foundations

Architecture and Training

Autoencoder Variants

Variational Autoencoders (VAEs)

Theoretical Perspectives

Practical Guide

Applications

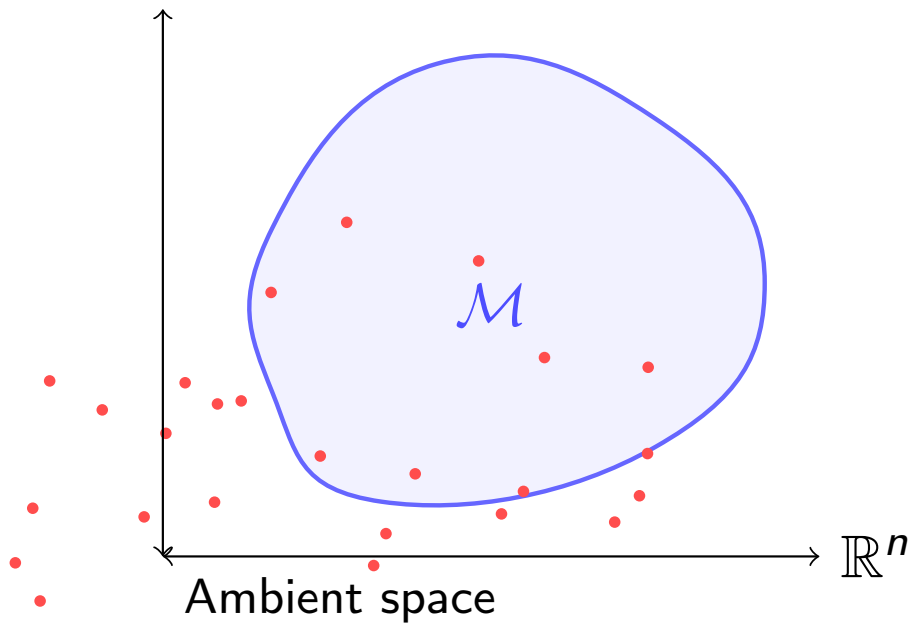
Conclusions

The Curse of High Dimensions

Problem: Real-world data lives in very high-dimensional spaces.

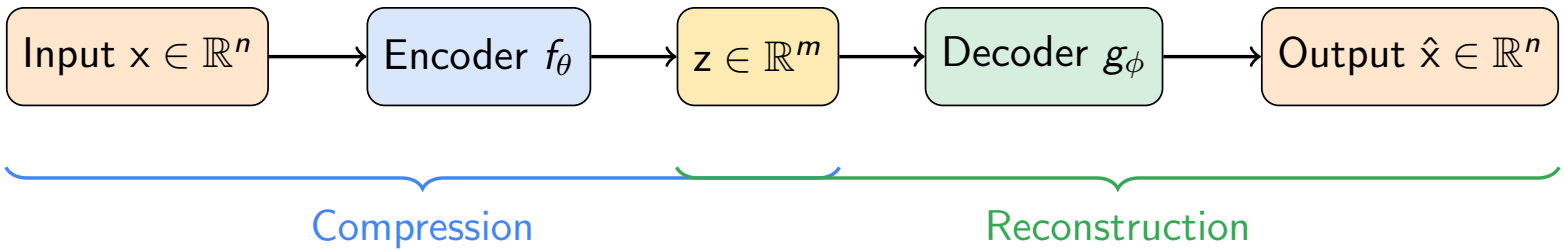
- A 256×256 RGB image has $256^2 \times 3 = 196{,}608$ dimensions.
- Yet most of that space is *empty* — natural images occupy a tiny sub-region.

Key insight: Data concentrates near a **low-dimensional manifold** $\mathcal{M} \subset \mathbb{R}^n$ with intrinsic dimension $m \ll n$.



What Is an Autoencoder? — The Elevator Pitch

An **autoencoder** is a neural network trained to *copy its input to its output* — through a **bottleneck**.



Because $m < n$, the network *cannot* simply memorise — it must discover **structure**.

Analogy: The Telephone Game

Think of it as a **communication** problem:

1. **Sender (Encoder):** Must compress a message into a few words.
2. **Channel (Bottleneck):** Limited bandwidth — only m numbers can pass.
3. **Receiver (Decoder):** Must reconstruct the full message from those few words.

The better the sender and receiver *agree on a code*, the lower the reconstruction error.

Key Takeaway

The bottleneck forces the autoencoder to learn a **compact, meaningful code** — not just memorise data.

Why Autoencoders? — Applications at a Glance

Representation Learning

- Dimensionality reduction
- Feature extraction
- Pre-training for downstream tasks

Data Cleaning

- Denoising
- Inpainting (filling missing regions)
- Super-resolution

Anomaly Detection

- High reconstruction error $\Rightarrow$ outlier
- Used in fraud detection, manufacturing QC

Generative Modeling

- VAEs generate new samples
- Latent space interpolation
- Drug discovery, image synthesis

Historical Context

- **1986** — Rumelhart, Hinton & Williams introduce backpropagation; early autoencoder ideas.
- **2006** — Hinton & Salakhutdinov show deep autoencoders outperform PCA for dimensionality reduction.
- **2008** — Vincent et al. introduce **Denoising Autoencoders**.
- **2011** — Rifai et al. introduce **Contractive Autoencoders**.
- **2013** — Kingma & Welling introduce the **Variational Autoencoder (VAE)**.
- **2017+** — VQ-VAE, β-VAE, and connections to diffusion models.

Perspective

Autoencoders are one of the oldest neural network ideas, yet they remain at the core of modern generative AI.

Formal Definition

An autoencoder consists of two parameterised functions:

$$\text{Encoder:} \quad f_\theta : \mathbb{R}^n \to \mathbb{R}^m, \quad \mathsf{z} = f_\theta(\mathsf{x}) \tag{1}$$

$$\text{Decoder:} \quad g_\phi : \mathbb{R}^m \to \mathbb{R}^n, \quad \hat{\mathsf{x}} = g_\phi(\mathsf{z}) \tag{2}$$

The full autoencoder is the **composition**:

$$h_{\theta,\phi}(\mathsf{x}) = (g_\phi \circ f_\theta)(\mathsf{x}) = g_\phi\big(f_\theta(\mathsf{x})\big) = \hat{\mathsf{x}}$$

Training objective:

$$\min_{\theta,\phi} \ \mathbb{E}_{\mathsf{x}\sim p_{\text{data}}}\big[\mathcal{L}(\mathsf{x},\, g_\phi(f_\theta(\mathsf{x})))\big]$$

where $\mathcal{L}$ is a **reconstruction loss**.

Reconstruction Losses

1. Mean Squared Error (MSE) — for continuous, real-valued data:

$$\mathcal{L}_{\text{MSE}}(\mathrm{x}, \hat{\mathrm{x}}) = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{x}_i)^2 = \frac{1}{n} \|\mathrm{x} - \hat{\mathrm{x}}\|_2^2$$

2. Binary Cross-Entropy (BCE) — when inputs are in $[0, 1]$ (e.g. normalised pixels):

$$\mathcal{L}_{\text{BCE}}(\mathrm{x}, \hat{\mathrm{x}}) = -\frac{1}{n} \sum_{i=1}^{n} \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right]$$

When to use which?

- MSE assumes **Gaussian** noise model: $p(\mathrm{x}|\hat{\mathrm{x}}) = \mathcal{N}(\hat{\mathrm{x}}, \sigma^2 I)$
- BCE assumes **Bernoulli**: each pixel is an independent coin flip with probability $\hat{x}_i$

The Probabilistic View of Reconstruction Loss

Reconstruction losses correspond to **maximum likelihood estimation**.

Given a decoder that defines $p_\phi(\mathrm{x}|\mathrm{z})$:

$$\max_\phi \log p_\phi(\mathrm{x}|\mathrm{z}) \quad \Longleftrightarrow \quad \min_\phi \mathcal{L}(\mathrm{x}, g_\phi(\mathrm{z}))$$

Likelihood model	**Distribution**	**Loss**
Gaussian	$\mathcal{N}(\hat{\mathrm{x}}, \sigma^2 I)$	MSE
Bernoulli	$\mathrm{Bern}(\hat{\mathrm{x}})$	Binary CE
Laplacian	$\mathrm{Laplace}(\hat{\mathrm{x}}, b)$	L_1 (MAE)

Intuition: Choosing a loss = choosing what kind of "noise" you expect around the reconstruction.

Connection to PCA

Theorem (Baldi & Hornik, 1989)

A **single-hidden-layer linear autoencoder** trained with MSE learns the same subspace as PCA.

Setup: Encoder $\mathrm{z} = W_e \mathrm{x}$, Decoder $\hat{\mathrm{x}} = W_d \mathrm{z}$.

Minimising $\|\mathrm{x} - W_d W_e \mathrm{x}\|^2$ over all data yields:

$$W_d W_e = U_m U_m^\top$$

where U_m contains the top-m eigenvectors of the data covariance Σ.

Nonlinear autoencoders generalise this: they can capture **curved manifolds** that PCA (a linear method) cannot.

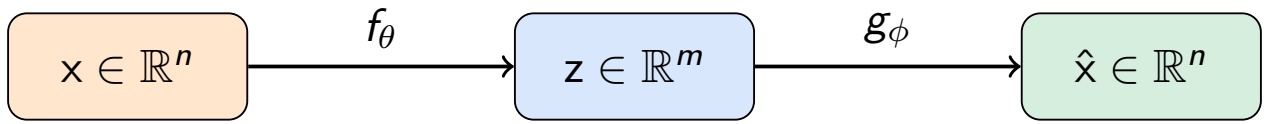
$\Rightarrow$ Autoencoders are **nonlinear PCA on steroids**.

Autoencoders as Approximate Bijections

Conceptually, the autoencoder learns an **approximate bijection**:

$$x \xrightarrow{f_\theta} z \xrightarrow{g_\phi \approx f_\theta^{-1}} \hat{x} \approx x$$

- Since $m < n$, the mapping *cannot* be truly bijective — information is irreversibly lost.
- The encoder projects onto a manifold; the decoder "lifts" back.
- Quality depends on how well the bottleneck dimension m matches the **intrinsic dimensionality** of the data.



$m \ll n$ (bottleneck)

Information Bottleneck Perspective

The autoencoder optimises an implicit **information bottleneck**:

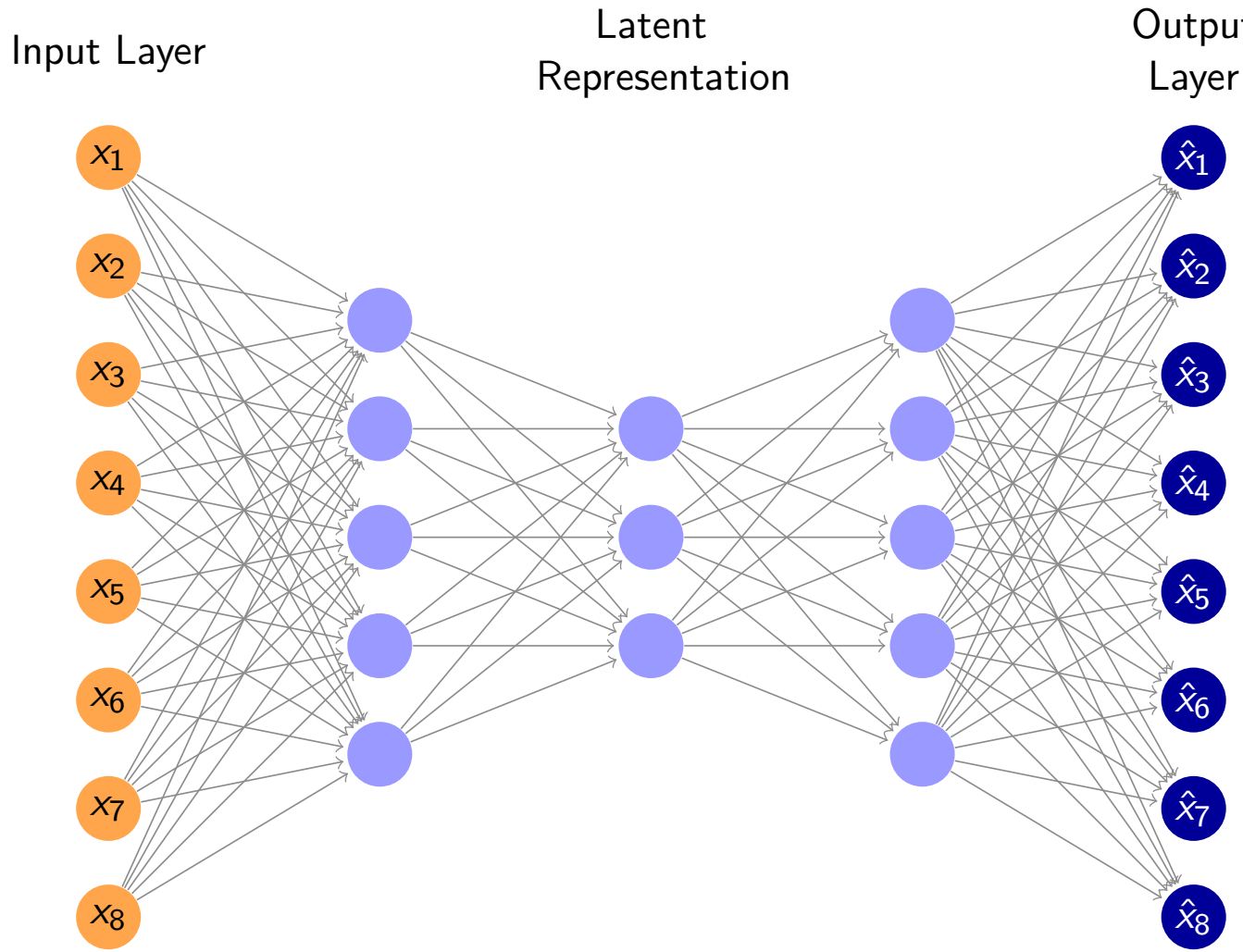
$$\min \quad \underbrace{I(x; z)}_{\text{compression}} \qquad \text{s.t.} \qquad \underbrace{I(z; x)}_{\text{sufficiency}} \geq I_0$$

In words:

- **Compress:** The code z should be as simple as possible (small m, regularisation).
- **Preserve:** But z must retain enough information to reconstruct x.

This tension between **compression** and **reconstruction fidelity** is the engine that drives meaningful representation learning.

Basic Autoencoder Architecture



Note the **hourglass shape**: $8 \to 5 \to 3 \to 5 \to 8$. The narrowest layer ($m = 3$) is the **bottleneck**.

Layer-by-Layer: What Happens Inside

Encoder (one hidden layer example):

$$h_1 = \sigma(W_1 x + b_1) \quad \text{(hidden layer)} \tag{3}$$

$$z = \sigma(W_2 h_1 + b_2) \quad \text{(latent code)} \tag{4}$$

Decoder (mirror structure):

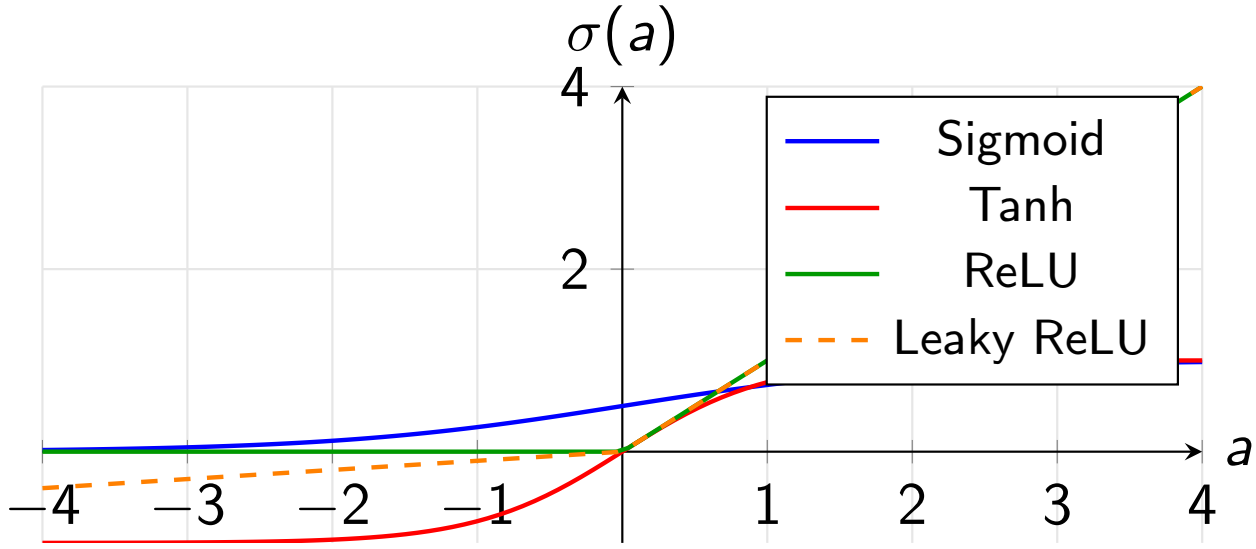
$$h_3 = \sigma(W_3 z + b_3) \tag{5}$$

$$\hat{x} = \sigma_{\text{out}}(W_4 h_3 + b_4) \tag{6}$$

Where σ is a nonlinear activation and σ_{out} matches the data range:

- **Sigmoid** if $x \in [0, 1]^n$ (normalised images)
- **Linear** if $x \in \mathbb{R}^n$ (general continuous data)
- **Tanh** if $x \in [-1, 1]^n$

Activation Functions Refresher



ReLU is the default for hidden layers. Use **Sigmoid/Tanh** at the output to match data range.

Tied Weights

A common trick: set decoder weights as the **transpose** of encoder weights.

$$W_d = W_e^\top$$

Benefits:

- Halves the number of parameters $\Rightarrow$ acts as **regularisation**.
- Enforces a geometric consistency: encoding and decoding use the "same directions."
- Can speed up training and reduce overfitting on small datasets.

Downside: Reduces model flexibility. For complex data, untied weights often perform better.

Training Procedure

1. **Forward pass:** Compute $\hat{\mathrm{x}} = g_\phi(f_\theta(\mathrm{x}))$.
2. **Compute loss:** $\mathcal{L}(\mathrm{x}, \hat{\mathrm{x}})$.
3. **Backward pass:** Compute gradients $\nabla_\theta \mathcal{L}$ and $\nabla_\phi \mathcal{L}$ via backpropagation through the *entire* network.
4. **Update:** $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}, \quad \phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}$.

Important

Encoder and decoder are trained **jointly, end-to-end**. Gradients flow through the bottleneck — this is what forces z to be informative.

Regularisation Techniques

Without regularisation, a powerful autoencoder can learn the **identity function** and memorise data.

Explicit regularisation:

- L_2 weight decay: $\mathcal{L} + \lambda \|\theta\|^2$
- Dropout (randomly zero activations)
- Early stopping

Implicit regularisation:

- Small bottleneck dimension m
- Batch normalisation
- Data augmentation
- Noise injection (denoising)

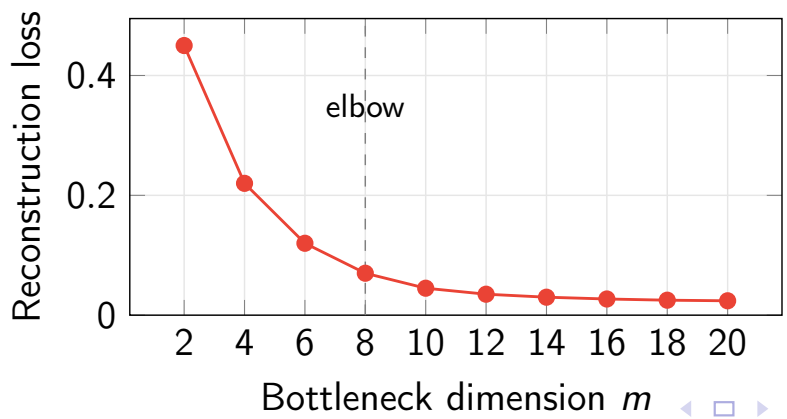
The bottleneck itself is the **strongest** regulariser — it is an *information bottleneck*.

Choosing the Bottleneck Dimension m

- **Too small:** Underfitting — not enough capacity to represent the data.
- **Too large:** Overfitting — the network can memorise without learning structure.
- **Just right:** Matches the **intrinsic dimensionality** of the data manifold.

How to choose m in practice:

1. Plot reconstruction loss vs. m — look for the **elbow**.
2. Use PCA as a baseline: how many principal components explain 95% of variance?
3. Cross-validate on a downstream task if features will be reused.



Optimizers

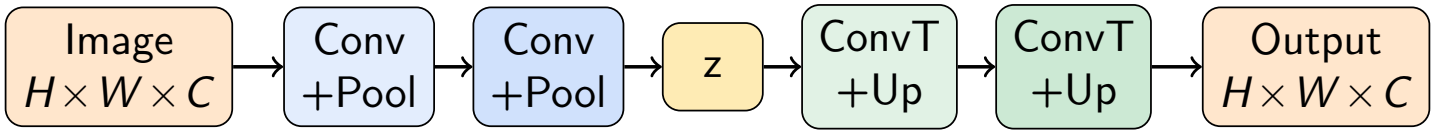
Optimizer	Notes
SGD + Momentum	Robust, but requires careful LR tuning
Adam	Adaptive LR; default choice for most autoencoder work
AdamW	Adam + decoupled weight decay; often better generalisation
RMSProp	Good for non-stationary objectives

Typical learning rates:

- Adam: $\eta \in [10^{-4}, 10^{-3}]$
- SGD: $\eta \in [10^{-2}, 10^{-1}]$ with decay schedule

Convolutional Autoencoders

For **image data**, replace fully-connected layers with convolutions:

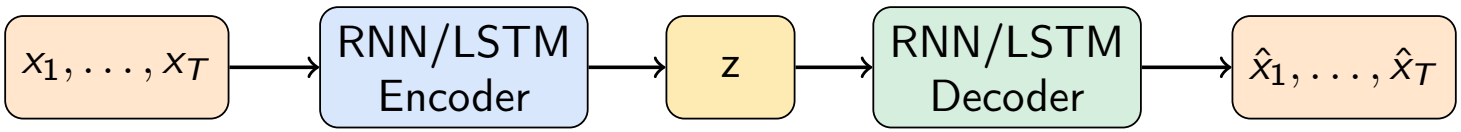


Key ideas:

- Encoder uses **strided convolutions** or pooling to downsample.
- Decoder uses **transposed convolutions** (or upsample + conv) to upsample.
- Exploits **spatial structure** — far fewer parameters than fully-connected.
- Skip connections (as in U-Net) can improve reconstruction quality.

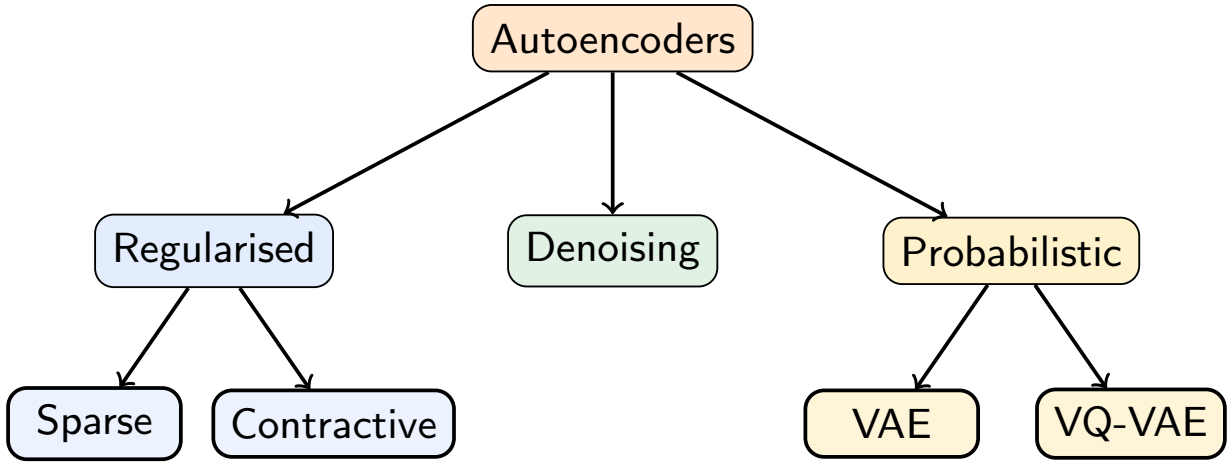
Recurrent and Sequence Autoencoders

For **sequential data** (text, time-series, audio):



- The encoder reads the full sequence and compresses it into a fixed-length vector z.
- The decoder unrolls z back into a sequence.
- Foundation of **seq2seq** models (precursor to Transformers).

Taxonomy of Autoencoders



Each variant adds a different **inductive bias** to the basic autoencoder framework.

Undercomplete vs Overcomplete Autoencoders

Undercomplete ($m < n$):

- Bottleneck forces compression.
- The "classic" setup.
- Regularised by architecture alone.

Overcomplete ($m \geq n$):

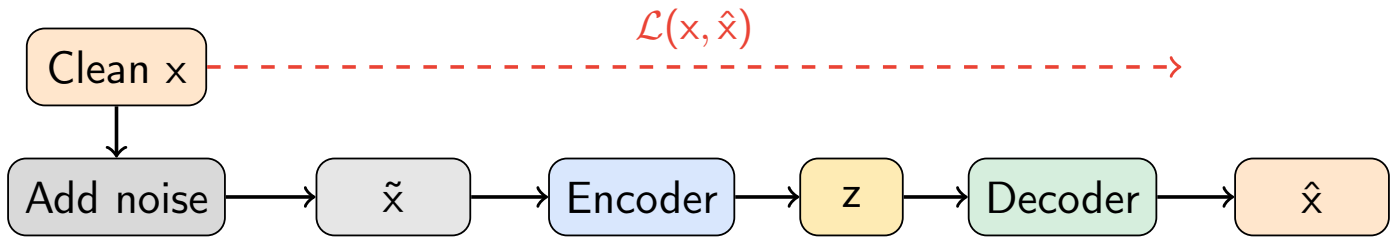
- Latent dim $\geq$ input dim.
- Can trivially learn identity!
- **Requires** explicit regularisation (sparsity, contractive penalty, noise).

Surprising Fact

Overcomplete autoencoders with proper regularisation can learn **better** features than undercomplete ones — they have more capacity to represent complex structure.

Denoising Autoencoder (DAE)

Idea: Corrupt the input, then train to reconstruct the *clean* original.



Common noise types:

- **Masking noise**: Randomly set a fraction of inputs to 0.
- **Gaussian noise**: $\tilde{x} = x + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I)$.
- **Salt-and-pepper noise**: Randomly flip values to min or max.

DAE: Why Does Noise Help?

- The model cannot simply copy $\tilde{x}$ to $\hat{x}$ — the noise is **unpredictable**.
- To reconstruct the clean signal, the encoder must learn the **underlying structure** of the data, not surface-level correlations.
- Formally, Vincent et al. (2008) showed that a DAE learns to estimate the **score function**:

$$\hat{x} - \tilde{x} \approx \sigma^2 \nabla_{\tilde{x}} \log p(\tilde{x})$$

i.e., the model learns the gradient of the log-density — pointing *towards* the data manifold.

Connection to Score Matching

This insight directly connects denoising autoencoders to **score-based diffusion models** — one of the foundations of modern image generation (DALL-E, Stable Diffusion).

Sparse Autoencoder

Goal: Only a few latent units should be "active" for any given input.

Objective:

$$\mathcal{L}_{\text{total}} = \underbrace{\|x - \hat{x}\|^2}_{\text{reconstruction}} + \underbrace{\beta \sum_{j=1}^{m} \text{KL}(\rho \,\|\, \hat{\rho}_j)}_{\text{sparsity penalty}}$$

where $\hat{\rho}_j = \frac{1}{N} \sum_{i=1}^{N} z_j^{(i)}$ is the average activation of unit j, and ρ is a small target (e.g. $\rho = 0.05$).

KL divergence as sparsity penalty:

$$\text{KL}(\rho \| \hat{\rho}) = \rho \log \frac{\rho}{\hat{\rho}} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}}$$

Intuition: Each input is represented by a **small subset** of features — like a dictionary where only a few words are used per sentence.

Sparse AE: Alternative Penalties

The KL penalty is not the only way to enforce sparsity:

Penalty	**Formula**
KL divergence	$\sum_j \mathrm{KL}(\rho \Vert \hat{\rho}_j)$
L_1 on activations	$\lambda \sum_j \lvert z_j \rvert$
Winner-take-all	Keep top-k activations, zero the rest
Lifetime sparsity	Each unit active for $\leq p\%$ of data

L_1 penalty is simplest; KL gives more control; winner-take-all is common in recent work (e.g. sparse autoencoders for **mechanistic interpretability** of LLMs).

Contractive Autoencoder (CAE)

Idea: Penalise the encoder for being too sensitive to small input perturbations.

Objective:

$$\mathcal{L}_{\text{CAE}} = \|\mathrm{x} - \hat{\mathrm{x}}\|^2 + \lambda \underbrace{\left\| \frac{\partial f_\theta(\mathrm{x})}{\partial \mathrm{x}} \right\|_F^2}_{\text{Frobenius norm of Jacobian}}$$

The Jacobian $J = \frac{\partial \mathrm{z}}{\partial \mathrm{x}} \in \mathbb{R}^{m \times n}$ measures how much each output changes w.r.t. each input.

Intuition:

- A small $\|J\|_F$ means the encoding is **locally invariant** — nearby inputs map to similar codes.
- The representation is **robust** to small perturbations (noise, irrelevant variations).
- Geometrically: the encoder "flattens" the mapping around the data manifold.

Comparing Regularised Autoencoders

	Denoising	**Sparse**	**Contractive**
Regularisation	Input noise	Activity penalty	Jacobian penalty
Target	Robustness to noise	Disentangled codes	Local invariance
Overcomplete?	Yes	Yes	Yes
Comp. cost	Low	Low–Medium	High (Jacobian)
Best for	Denoising, pretraining	Interpretability	Manifold learning

Rifai et al. (2011)

Showed that DAE and CAE are closely related: denoising with small Gaussian noise approximately minimises the same Jacobian penalty.

From Deterministic to Probabilistic

Problem with vanilla autoencoders:

- The latent space may be **irregular** — gaps, discontinuities.
- You can't sample z from the latent space and get a meaningful output.
- Not a true **generative model**.

Solution (Kingma & Welling, 2013): Make it **probabilistic**.

- Instead of mapping $\mathrm{x} \mapsto \mathrm{z}$ (a point), map $\mathrm{x} \mapsto q(\mathrm{z}|\mathrm{x})$ (a **distribution**).
- Regularise $q(\mathrm{z}|\mathrm{x})$ to be close to a simple prior $p(\mathrm{z}) = \mathcal{N}(0, I)$.
- Now we *can* sample from $p(\mathrm{z})$ and decode to generate new data.

VAE: The Generative Story

The VAE defines a **generative model**:

1. Sample a latent code: $\mathrm{z} \sim p(\mathrm{z}) = \mathcal{N}(0, I)$
2. Generate data: $\mathrm{x} \sim p_\phi(\mathrm{x}|\mathrm{z})$ (decoder)

The marginal likelihood of data:

$$p_\phi(\mathrm{x}) = \int p_\phi(\mathrm{x}|\mathrm{z})\, p(\mathrm{z})\, d\mathrm{z}$$

This integral is **intractable** (we can't enumerate all possible z's).

$\Rightarrow$ We need a clever trick: **variational inference**.

Deriving the ELBO

Introduce an **approximate posterior** $q_\theta(z|x)$ (the encoder).

$$\begin{aligned}\log p_\phi(x) &= \log \int p_\phi(x|z)\, p(z)\, dz \\ &\geq \underbrace{\mathbb{E}_{q_\theta(z|x)}\big[\log p_\phi(x|z)\big]}_{\text{reconstruction term}} - \underbrace{\mathrm{KL}\big(q_\theta(z|x) \,\|\, p(z)\big)}_{\text{regularisation term}} \qquad (7)\end{aligned}$$

This lower bound is called the **Evidence Lower BOund (ELBO)**.

Intuition:

- **Reconstruction term:** "How well can I reconstruct x from z?"
- **KL term:** "How close is my encoder distribution to the prior?"

VAE Loss Function

In practice, for a single data point:

$$\boxed{\mathcal{L}_{\text{VAE}} = \underbrace{-\mathbb{E}_{q_\theta(\mathrm{z}|\mathrm{x})}\left[\log p_\phi(\mathrm{x}|\mathrm{z})\right]}_{\approx \text{ MSE or BCE}} + \underbrace{\mathrm{KL}\left(q_\theta(\mathrm{z}|\mathrm{x}) \,\|\, \mathcal{N}(0, I)\right)}_{\text{closed-form}}}$$

For Gaussian encoder $q_\theta(\mathrm{z}|\mathrm{x}) = \mathcal{N}(\boldsymbol{\mu}, \mathrm{diag}(\boldsymbol{\sigma}^2))$:

$$\mathrm{KL} = -\frac{1}{2}\sum_{j=1}^{m}\left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

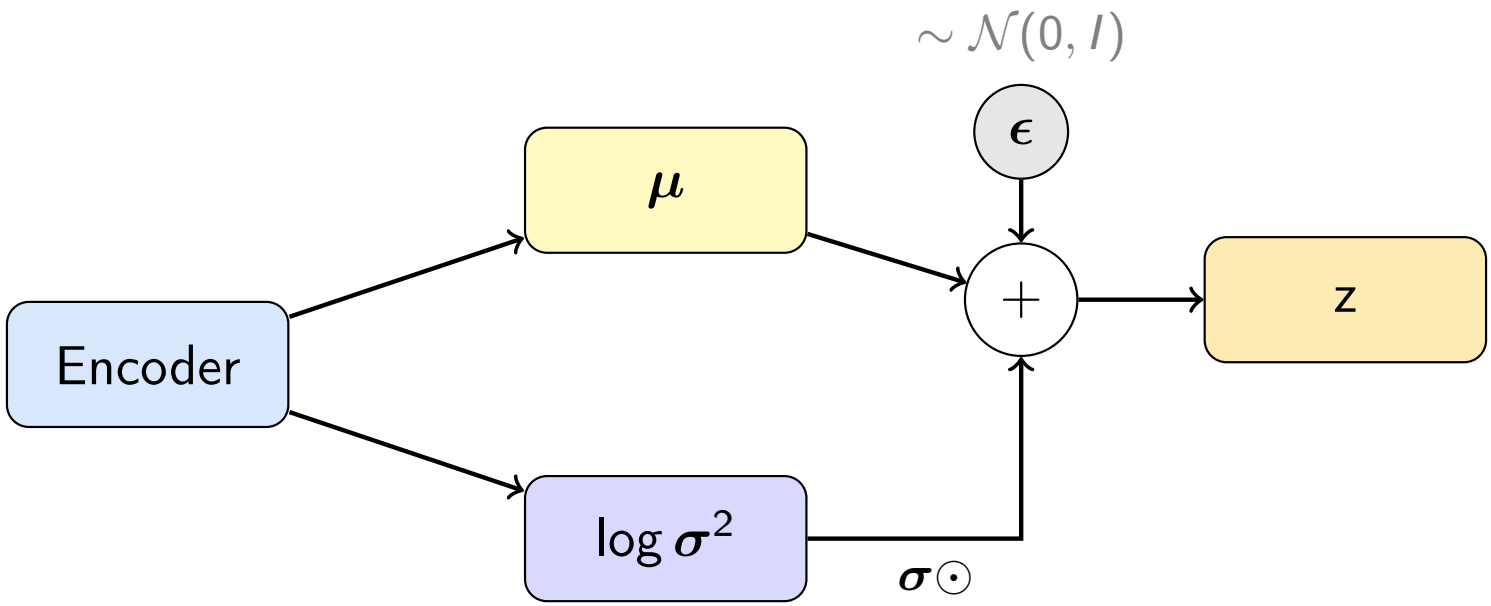
This has a **clean closed-form solution** — no sampling needed for the KL term!

The Reparameterisation Trick

Problem: We need to backpropagate through $z \sim q_\theta(z|x)$, but sampling is not differentiable.

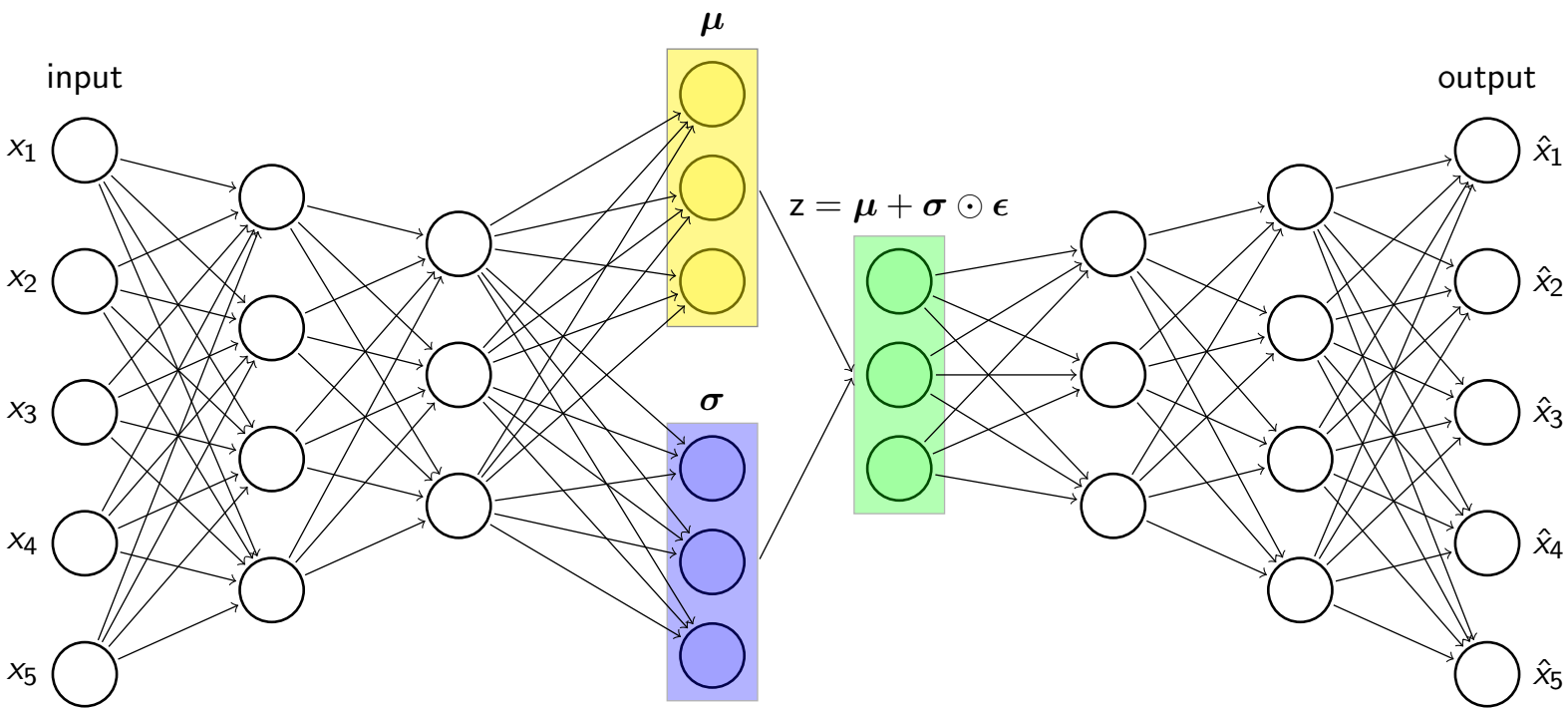
Solution: Reparameterise the random variable:

$$z = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$$



Now $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are deterministic outputs of the encoder $\Rightarrow$ gradients flow normally.

VAE Architecture



VAE: Latent Space Properties

The KL penalty ensures the latent space is **smooth and regular**:

Vanilla AE latent space:

- Irregular, with gaps
- Interpolation gives garbage
- Can't sample meaningfully

VAE latent space:

- Smooth, continuous
- Interpolation is meaningful
- Sampling gives realistic outputs

Why? The KL term pulls all $q(\mathrm{z}|\mathrm{x})$ towards $\mathcal{N}(0, I)$, filling in the gaps and ensuring nearby z's decode to similar x's.

The KL–Reconstruction Trade-off

The two ELBO terms **compete**:

- **Low KL, high reconstruction loss:** Encoder collapses everything to $\mathcal{N}(0, I)$ — ignores the input. This is called **posterior collapse**.
- **Low reconstruction loss, high KL:** Great reconstruction but irregular latent space — back to a vanilla AE.

Common mitigation strategies:

1. **KL annealing**: Gradually increase the KL weight from 0 to 1 during training.
2. β-**VAE**: Use weight β on the KL term — $\beta > 1$ encourages disentanglement, $\beta < 1$ prioritises reconstruction.
3. **Free bits**: Allow a minimum amount of KL per dimension before penalising.

β-VAE and Disentanglement

β-VAE (Higgins et al., 2017) modifies the ELBO:

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}[\text{recon. loss}] + \beta \cdot \text{KL}(q_\theta(\text{z}|\text{x}) \| p(\text{z}))$$

With $\beta > 1$, the model is **more strongly pushed** to compress, yielding a latent space where:

- Each z_j captures a **single independent factor** of variation.
- Example: for faces, one dimension encodes pose, another encodes hair colour, etc.

Trade-off: Higher $\beta \Rightarrow$ more disentangled but blurrier reconstructions.

VQ-VAE: Discrete Latent Spaces

Vector-Quantised VAE (van den Oord et al., 2017) replaces continuous z with a **discrete codebook**.

1. Encoder outputs continuous z_e.
2. Quantise: find nearest codebook vector $e_k = \arg\min_k \|z_e - e_k\|$.
3. Decoder receives e_k (discrete!).

Loss:

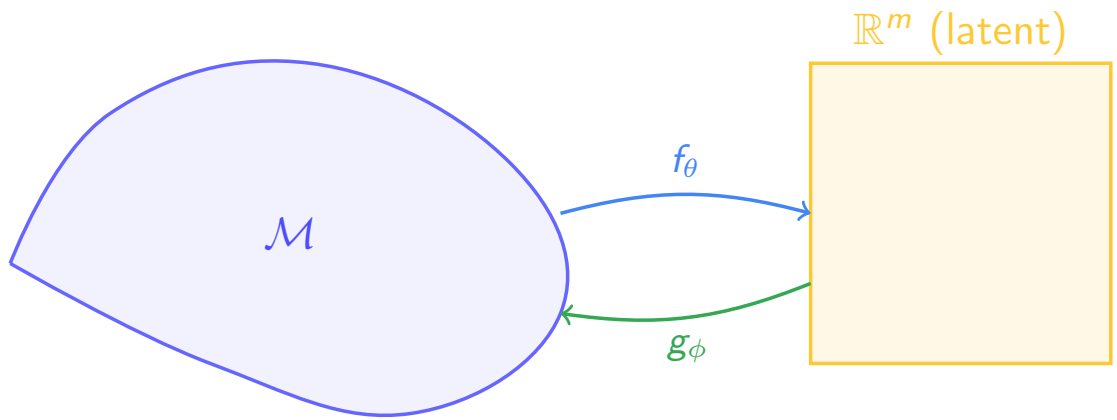
$$\mathcal{L} = \|x - \hat{x}\|^2 + \|\text{sg}[z_e] - e_k\|^2 + \beta\|z_e - \text{sg}[e_k]\|^2$$

where sg[·] is the stop-gradient operator.

Why discrete? Avoids posterior collapse; natural fit for token-based generation. VQ-VAE is the backbone of many modern image and audio generation systems.

Manifold Learning Perspective

- Data lies on a **low-dimensional manifold** $\mathcal{M} \subset \mathbb{R}^n$ of intrinsic dimension d.
- The encoder learns a **chart**: a local coordinate system on $\mathcal{M}$.
- The decoder learns the **inverse chart**: mapping coordinates back to the ambient space.



Approximation Power and Depth

Universal Approximation

A sufficiently wide autoencoder with one hidden layer in encoder and decoder can approximate any continuous mapping to arbitrary precision.

But depth matters in practice:

- **Shallow** (1 hidden layer): Can approximate anything, but may need exponentially many neurons.
- **Deep** (multiple layers): Can represent hierarchical features with far fewer parameters.
- Hinton & Salakhutdinov (2006): Deep autoencoders dramatically outperform shallow ones and PCA.

Analogy: Depth lets the network build features *on top of* features — edges $\rightarrow$ textures $\rightarrow$ parts $\rightarrow$ objects.

When Autoencoders Fail

Failure modes to watch for:

1. **Trivial identity mapping:** Bottleneck too wide, no regularisation $\Rightarrow$ memorisation.
2. **Mode collapse:** All inputs map to the same point in latent space.
3. **Blurry reconstructions:** MSE loss averages over modes $\Rightarrow$ blurriness (especially for images).
4. **Posterior collapse (VAE):** Decoder ignores z entirely; KL goes to 0.
5. **Latent space holes:** Regions of latent space that decode to nonsense (vanilla AE).

Remedies: Proper bottleneck size, regularisation, perceptual losses, KL annealing, architecture choice.

Recipe for Building an Autoencoder

1. **Understand your data:** Images? Text? Tabular? Time-series?
2. **Choose architecture:**
   - Images $\rightarrow$ Convolutional AE
   - Sequences $\rightarrow$ Recurrent or Transformer AE
   - Tabular $\rightarrow$ Fully-connected AE
3. **Choose loss:** MSE for continuous, BCE for $[0, 1]$ data.
4. **Set bottleneck m:** Start with PCA estimate, then tune.
5. **Add regularisation:** Dropout, weight decay, or use a variant (DAE, sparse, VAE).
6. **Train:** Adam, batch size 64–256, monitor both train and validation loss.
7. **Evaluate:** Reconstruction quality, latent space visualisation (t-SNE/UMAP), downstream task performance.

Data Preprocessing

- **Normalisation** is critical:
  - Scale to $[0, 1]$ if using sigmoid output + BCE loss.
  - Standardise to zero mean, unit variance if using linear output + MSE.
- **Data augmentation** (for images): flips, crops, colour jitter — acts as implicit regularisation.
- **Handle missing data:** Autoencoders can learn to impute, but mask the loss on missing entries.

Common Pitfall

Using BCE loss with data not in $[0, 1]$, or MSE loss with a sigmoid output and data in $\{0, 1\}$. Always match loss to data range and output activation.

Evaluating Autoencoders

Reconstruction quality:

- MSE / MAE on held-out test data.
- Visual inspection of input vs. output (for images).
- SSIM (Structural Similarity) for perceptual quality.

Latent space quality:

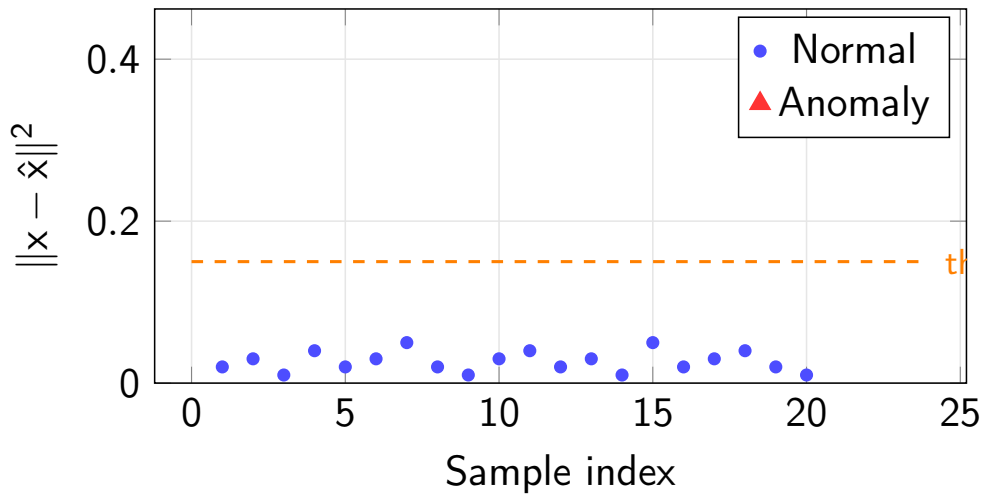
- Visualise with t-SNE or UMAP — do classes cluster?
- Interpolation: smooth transition between two points in latent space?
- For VAEs: FID (Fréchet Inception Distance) for generation quality.

Downstream performance:

- Train a classifier on z — are the features useful?
- Anomaly detection: ROC-AUC using reconstruction error as score.

Application: Anomaly Detection

Principle: Train on *normal* data only. Anomalies produce **high reconstruction error**.



Used in: credit card fraud, manufacturing defect detection, network intrusion, medical diagnosis.

Application: Image Denoising and Inpainting

Denoising: Train a DAE on noisy $\rightarrow$ clean pairs. At test time, feed noisy image, get clean output.

Inpainting: Mask out a region; the autoencoder "fills in" the missing area by reconstructing the full image from the visible parts.

Key applications:

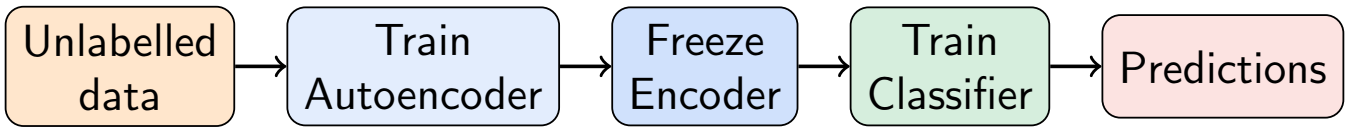
- Medical imaging (MRI/CT denoising to reduce radiation dose)
- Photo restoration (removing scratches, damage)
- Video compression (reconstructing dropped frames)

Modern Connection

Denoising autoencoders are the conceptual ancestor of **diffusion models** — the technology behind DALL-E, Stable Diffusion, and Midjourney.

Application: Representation Learning and Transfer

Strategy: Train an autoencoder on unlabelled data, then use the encoder as a **feature extractor**.



Step 1: Unsupervised Step 2: Supervised

Why it works: The autoencoder learns general features (edges, textures, patterns) that transfer well to specific tasks, even with **very few labelled examples**.

Application: Latent Space Interpolation

VAEs enable smooth generation by **interpolating** in latent space:

$$z_\alpha = (1 - \alpha)\, z_A + \alpha\, z_B, \quad \alpha \in [0, 1]$$

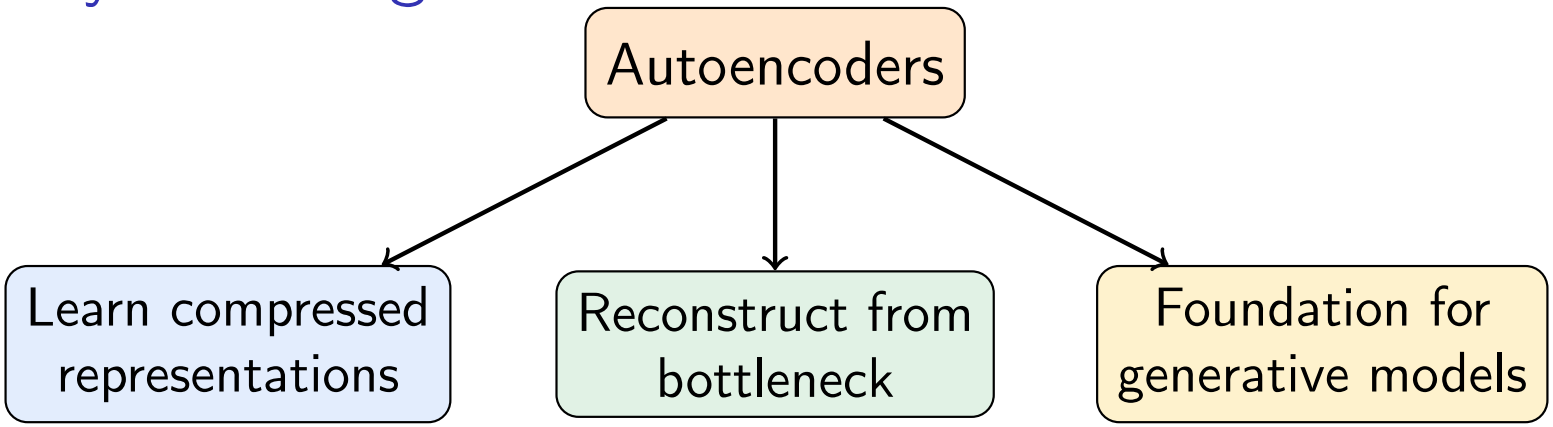
Decoding z_α produces a **smooth morph** between x_A and x_B.

Applications:

- Face morphing and attribute editing (smile $\rightarrow$ neutral)
- Drug discovery: interpolate between molecular structures
- Creative tools: blend styles, generate variations
- **Latent arithmetic:**

  $z_{\text{man with glasses}} - z_{\text{man}} + z_{\text{woman}} \approx z_{\text{woman with glasses}}$

Summary: The Big Picture



Key Takeaways

1. Autoencoders learn **data-driven compression** through reconstruction.
2. The **bottleneck** is the key ingredient forcing meaningful representations.
3. Variants (DAE, Sparse, Contractive, VAE, VQ-VAE) add different inductive biases.
4. VAEs bridge autoencoders and **generative modelling**.
5. Modern generative AI (diffusion models) traces its intellectual lineage to autoencoders.

What's Next?

In this course:

- Generative Adversarial Networks (GANs)
- Normalising Flows
- Diffusion Models

Further exploration:

- Wasserstein Autoencoders (WAE)
- Adversarial Autoencoders (AAE)
- Hierarchical VAEs (NVAE, VDVAE)
- Autoencoders for self-supervised learning (MAE — Masked Autoencoders)

References and Further Reading

- Hinton, G. E. & Salakhutdinov, R. R. *Reducing the dimensionality of data with neural networks.* Science, 2006.
- Vincent, P. et al. *Extracting and composing robust features with denoising autoencoders.* ICML, 2008.
- Rifai, S. et al. *Contractive auto-encoders.* ICML, 2011.
- Bengio, Y. et al. *Representation Learning: A Review and New Perspectives.* IEEE TPAMI, 2013.
- Kingma, D. P. & Welling, M. *Auto-Encoding Variational Bayes.* ICLR, 2014.
- Higgins, I. et al. *β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework.* ICLR, 2017.
- van den Oord, A. et al. *Neural Discrete Representation Learning (VQ-VAE).* NeurIPS, 2017.
- Baldi, P. & Hornik, K. *Neural Networks and Principal Component Analysis.* Neural Networks, 1989.
- He, K. et al. *Masked Autoencoders Are Scalable Vision Learners.* CVPR, 2022.

Questions?

Thank you!